## BROTAR

# Manual de instalación y correo

Guía para instalar una copia propia, preparar PostgreSQL, iniciar frontend y backend e importar el correo autorizado sin copiar credenciales de base.

Proyecto de Sistemas II

Ricardo Núñez del Prado Justiniano

Santiago Rocha Mejia

Alison Pizarro Apaza

Corte DEV 30 de octubre de 2026

Entrega S3-20 en DEV autorizada por Ricardo

Recorrido S3-21 aceptado por Ricardo para el alcance académico DEMO

Pagos y expedientes de prueba. Condiciones provisionales del equipo. La contraseña Gmail se entrega en un paquete privado separado; no está en estos documentos.

Entrega del MVP académico de crowdfunding para **DEV**, 30/10/2026. Integra los tres sprints y la aceptación de Ricardo del recorrido S3-21. **Pagos, QR, destinos y operaciones económicas son DEMO**; las condiciones legales son provisionales del equipo. El correo Gmail funciona con configuración privada autorizada.

Publicación de S3-20 autorizada por Ricardo en DEV. Registro del corte y comprobaciones. La revisión manual adicional del equipo se organiza en tres recorridos que cubren las 73 rutas; sus resultados se registran aparte y no se presuponen aprobados. El paquete privado de correo se comparte por separado; no está en Git.

# 1. Qué incluye

Descubrimiento público, cuenta y sesiones, recuperación y verificación de correo, perfiles, organizaciones y representación, KYC/KYB sintético, constructor completo, revisión/corrección/publicación, aportes y comprobantes DEMO, favoritos, avances y recompensas, rendición, ciclo/cierre, desembolsos, devoluciones/disputas, soporte, catálogos, parámetros, permisos y auditoría.

**40 migraciones, 215 rutas HTTP y 73 patrones de rutas de interfaz.** Fuente del estado: planificación local, con el estado de preparación/publicación separado del histórico de Trello. Entrega final y límites.

# 2. Requisitos

| Componente | Versión y función |
|---|---|
| Node.js | 24, mínimo 24.13.0 y menor que 25 |
| Bun | 1.4.2, frontend en la raíz |
| pnpm | 11.19.0 mediante Corepack, backend en backend/ |
| Docker Desktop | Motor Linux operativo; ejecuta solo PostgreSQL 18.6 |
| Git | Para clonar y gestionar ramas; el ZIP local también permite instalar |

Infraestructura vigente: **infra/postgres-v2/**. **infra/postgres/** contiene antecedentes y consultas auxiliares que usa el verificador; no levantar su contenedor. NestJS y Vite corren en el anfitrión.

| Servicio normal | Dirección |
|---|---|
| PostgreSQL Docker | 127.0.0.1:15433 hacia 5432 dentro del contenedor |
| API | http://127.0.0.1:3000/api |
| Interfaz | http://127.0.0.1:5173 |

Comprobar en PowerShell: `git --version`, `node --version`, `bun --version`, `docker version`; dentro de backend: `corepack pnpm --version`. Docker debe mostrar cliente y servidor.

# 3. Primera instalación completa

En una carpeta nueva, obtener DEV en PowerShell:

```
git clone --branch DEV --single-branch `
  https://github.com/luisrocha159/CrowFunding-Brotar.git Brotar
cd Brotar
```

Si ya se tiene el código, abrir PowerShell en su raíz y seguir la sección 5 para actualizar una instalación existente. **Comprobar exit 0 antes de continuar con cada comando.** No copiar .env de otra computadora.

```
bun install --frozen-lockfile
cd backend
corepack pnpm install --frozen-lockfile
```

Si ya existe otra copia de Brotar en la PC y esta será una instalación nueva aislada, elegir un proyecto y puerto libres **antes de install**, solo en esa consola:

```
$env:BROTAR_COMPOSE_PROJECT='brotar-equipo-prueba'
$env:BROTAR_DB_PORT='15457'
```

En una PC nueva no hace falta ese bloque: se usan brotar-provisional-v2 y 15433. Otra carpeta con el mismo proyecto Compose comparte base y volumen. Las variables no reemplazan una configuración V2 ya creada.

Desde backend, para una base nueva:

```
node scripts/adopt-provisional-v2.mjs install ../infra/postgres-v2/official-schema.sql
corepack pnpm run check
node scripts/migrate.mjs up
node scripts/prepare-sprint2.mjs
node scripts/migrate.mjs status
node scripts/verify-provisional-v2.mjs
node scripts/adopt-provisional-v2.mjs activate
```

Esperado: **40 migraciones, ninguna pendiente**, catálogo con cinco categorías provisionales activas e inventario válido. install restaura la línea base oficial únicamente para una base nueva; las migraciones y prepare-sprint2 cargan los catálogos/permisos. No ejecutar SQL adicional para las categorías. El healthcheck espera el servidor TCP definitivo de PostgreSQL.

install genera infra/postgres-v2/.env (administración) y .env.backend (aplicación limitada); activate genera backend/.env. No convertir backend/.env.example en una conexión operativa. El rol de la API es brotar_app, nunca postgres. Se conserva el SHA-256 del SQL oficial y no se sincroniza el esquema con el ORM.

**Solo tras esta primera activación**, añadir al backend/.env nuevo las opciones del MVP de pruebas con este comando PowerShell. No pegar comandos en el archivo:

```
Add-Content -LiteralPath .env -Value 'DEMO_PAYMENTS_ENABLED=true'
Add-Content -LiteralPath .env -Value 'PUBLIC_WEB_ORIGIN=http://127.0.0.1:5173'
Add-Content -LiteralPath .env -Value 'PASSWORD_RESET_LOCAL_FILE=false'
```

CAMPAIGN_ADMIN_PUBLICATION_ENABLED=true ya lo genera el instalador. En una copia existente, editar los valores actuales de estas claves; no añadir claves repetidas. DEMO_PAYMENTS_ENABLED debe habilitarse explícitamente para probar aportes y finanzas. Producción deniega el simulador.

Completar **correo real** con la sección 6 antes de probar recuperación/verificación. Desde backend arrancar:

```
corepack pnpm run dev
```

En otra terminal, desde la raíz:

```
bun run check
bun run dev
```

Abrir http://127.0.0.1:5173. /api/health/live comprueba proceso; **http://127.0.0.1:3000/api/health/ready debe responder 200** para acreditar esquema y permisos preparados.

# 4. Abrir otro día

Iniciar Docker Desktop. Desde la raíz:

```
docker compose --env-file infra/postgres-v2/.env `
  -f infra/postgres-v2/compose.yaml up -d --wait
cd backend
corepack pnpm run dev
```

En otra terminal de la raíz: `bun run dev`. Conservar .env, volumen Docker y backend/private/files. No repetir install ni activate para volver a abrir el proyecto.

# 5. Actualizar una copia existente

Guardar trabajo propio y revisar el diff. Respaldar base y archivos desde backend compilado con `node scripts/drill-recovery.mjs --with-files`; conserva respaldo privado y prueba recuperación aislada. Detener la API para actualizar. Desde una copia sin cambios propios pendientes, actualizar DEV sin forzar:

```
git status --short
git fetch origin
git switch DEV
git pull --ff-only origin DEV
```

Si hay trabajo propio o el pull no puede avanzar, conservarlo y revisar CONTRIBUTING; no hacer reset ni forzar. Iniciar Docker Desktop y, desde la raíz, actualizar dependencias y base existente:

```
docker compose --env-file infra/postgres-v2/.env `
  -f infra/postgres-v2/compose.yaml up -d --wait
bun install --frozen-lockfile
cd backend
corepack pnpm install --frozen-lockfile
corepack pnpm run check
node scripts/migrate.mjs up
node scripts/prepare-sprint2.mjs
node scripts/migrate.mjs status
node scripts/verify-provisional-v2.mjs
corepack pnpm run dev
```

El frontend se comprueba/arranca aparte desde la raíz. Conservar backend/.env propio y SMTP privado; **no repetir install/activate sobre una V2 migrada**, no restaurar SQL encima ni borrar el volumen. Para una copia muy antigua, revisar el respaldo y migraciones.

# 6. Correo Gmail y traslado privado al equipo

El repo contiene adaptador, importador y diagnóstico, **no la contraseña Gmail**. El paquete privado preparado para el equipo contiene solo siete variables de correo, guía y script de importación; se comparte fuera de Git/Trello. No copiar un .env completo ni credenciales de base ajenas.

Desde backend, después de instalar, compilar y activar su base propia:

```
node scripts/mail-config.mjs --import 'C:/ruta/privada/correo-equipo.env'
node --env-file-if-exists=.env scripts/check-mail.mjs
```

El segundo comando comprueba TLS y autenticación **sin enviar mensajes**. Para enviar una prueba únicamente a la cuenta remitente por decisión del equipo:

```
node --env-file-if-exists=.env scripts/check-mail.mjs --send-self-test
```

Configurar manualmente, si no se recibe archivo privado:

```
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=cuenta-autorizada@gmail.com
SMTP_FROM=cuenta-autorizada@gmail.com
SMTP_PASSWORD=CONTRASENA_DE_APLICACION_PRIVADA
PASSWORD_RESET_LOCAL_FILE=false
PUBLIC_WEB_ORIGIN=http://127.0.0.1:5173
```

La clave debe ser de aplicación, no la contraseña de la cuenta. USER y FROM coinciden. 587 usa STARTTLS; no desactivar certificados. Cambiar PUBLIC_WEB_ORIGIN y CORS_ORIGINS si cambia la dirección del frontend; reiniciar la API. Cada integrante prueba en su PC: 127.0.0.1 apunta a quien abre el enlace. No se publica una instalación local en Internet mediante SMTP.

Recuperar acceso envía al correo registrado, con respuesta uniforme aunque no exista o falle el proveedor. Enlace de recuperación: una hora, un uso, revoca sesiones previas al consumirse. Verificar correo se solicita desde Mi cuenta, con token de 30 minutos y un uso; abrirlo no confirma automáticamente. Los avisos opcionales exigen correo verificado y preferencia activada. No hay verificación SMS. Procedimiento y diagnóstico de correo.

# 7. Primera cuenta administrativa y roles

Las migraciones no copian usuarios ni crean un administrador con contraseña pública. Registrar una cuenta propia en /registro y, desde backend de su instalación local, conceder ADMIN explícitamente:

```
node scripts/grant-local-admin.mjs --email=tu-cuenta@gmail.com --confirm-local-admin
```

Reemplazar el correo por el de la cuenta ya registrada. No cambia contraseña, identidad ni verificaciones. ADMIN concede CREATOR provisionalmente y responsabilidades internas desde /administracion/personas. REVIEWER revisa, COMPLIANCE verifica expedientes, FINANCE opera dinero DEMO, SUPPORT atiende y AUDITOR consulta. ADMIN no hereda operación financiera; AUDITOR no muta aunque tenga otros roles.

# 8. Validación y demostración

Checks sin datos: `bun run check` en raíz y `corepack pnpm run check` en backend. Integraciones con escritura: **solo copia QA nueva**, proyecto Compose con qa, puerto/archivos propios y sin SMTP personal. Procedimiento completo S3-21. No activar ALLOW_DB_TEST_WRITES en la base de uso.

Caso integral aceptado: registro/contacto/KYC DEMO → campaña → corrección/revisión → publicación separada → aporte único → soporte → resultado/desembolso → avance/recompensa/rendición → cierre. **565 pruebas distintas aprobadas en S3-21**, no se suman repeticiones. Resultados y aceptación.

# 9. Documentos finales

- Entrega y comprobaciones S3-20.

- Publicación vigente en DEV y tres recorridos manuales del equipo.

- Manual de instalación y correo.

- Product Backlog final y fuente con 64 historias/67 tareas.

- Documento explicativo final.

- Pantallas finales del MVP, con inventario de rutas y capturas representativas reales, separado del prototipo histórico.

- Figma CrownFundingV3 de coordinación; accesible al comprobar la entrega, sin afirmar identidad visual completa.

- Contratos HTTP, reglas del MVP e índice.

# 10. Diagnóstico rápido

| Síntoma | Comprobación |
|---|---|
| No conecta API | API y frontend en sus carpetas, puertos correctos, ready 200, .env generado, BACKEND_URL si se cambia 3000 |
| Categorías vacías | migrate up → prepare-sprint2 → verify; no hardcodear frontend ni restaurar SQL |
| Contraseña brotar_app rechazada | Configuración de esta copia y contenedor coincidentes; no conectar al PostgreSQL local de 5432 |
| Correo no llega | PASSWORD_RESET_LOCAL_FILE=false, SMTP válido, check-mail, bandeja/spam, reinicio y origen correcto |
| No aparece campaña al público | Debe estar publicada; aprobar es acción distinta. Revisar estado/propiedad/permisos |
| Puerto ocupado | Identificar el servicio; elegir puerto y proyecto nuevos antes del primer install de otra copia |
| Docker no dispone de subred | Revisar redes propias sin uso; no usar prune global, reset o down -v para resolverlo |

Los valores de QA5198/3198/15448 de capturas son de ensayo, no los puertos normales. No versionar secretos, backups ni archivos privados. Trabajo del equipo. Los cortes anteriores permanecen en README histórico; no son instrucciones de la entrega vigente.